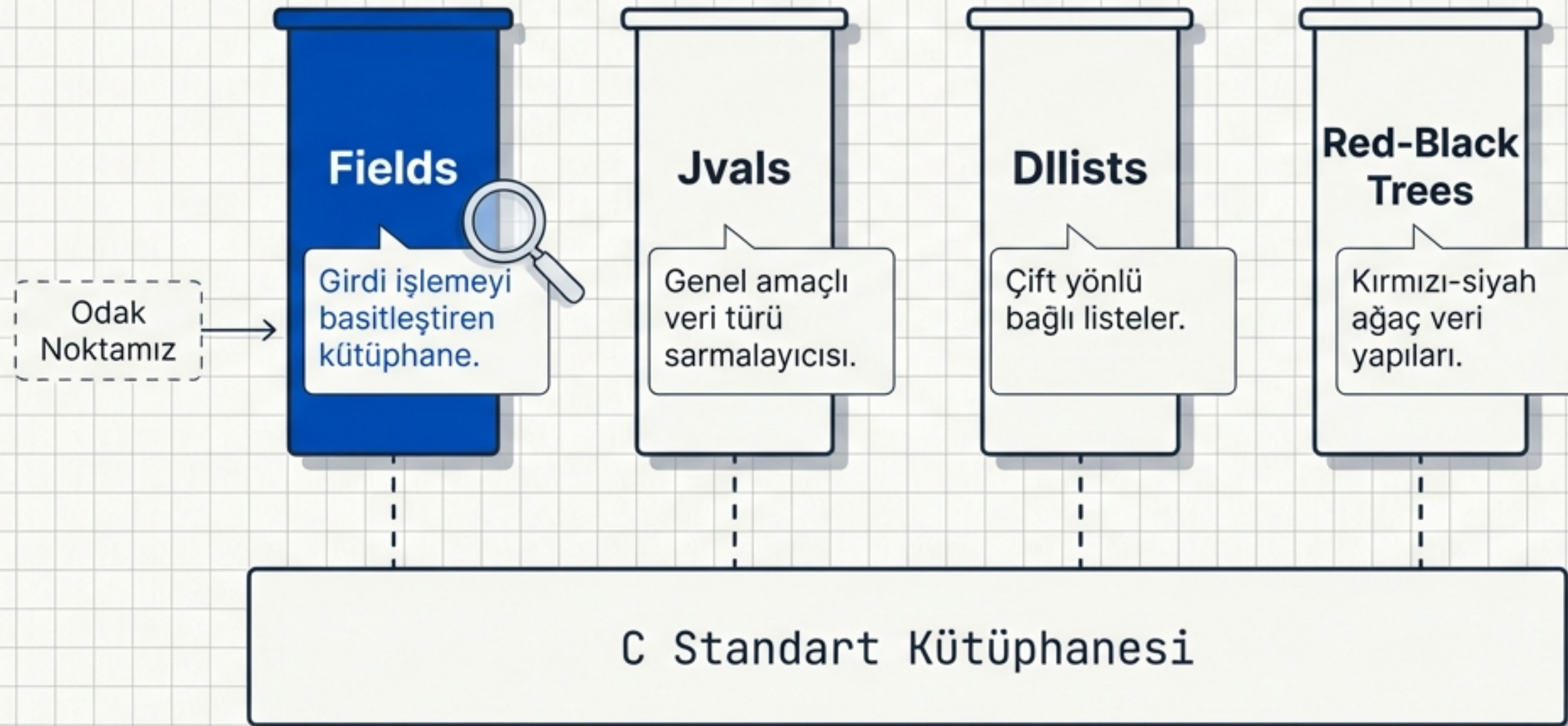


CS360 / BÖLÜM 6

Sistem Programlama: Gelişmiş Girdi İşleme ve Bellek Yönetimi

Dr. Öğr. Üyesi Abdullah Sevin

Libfdr: Sistem Programlama Cephaneliği



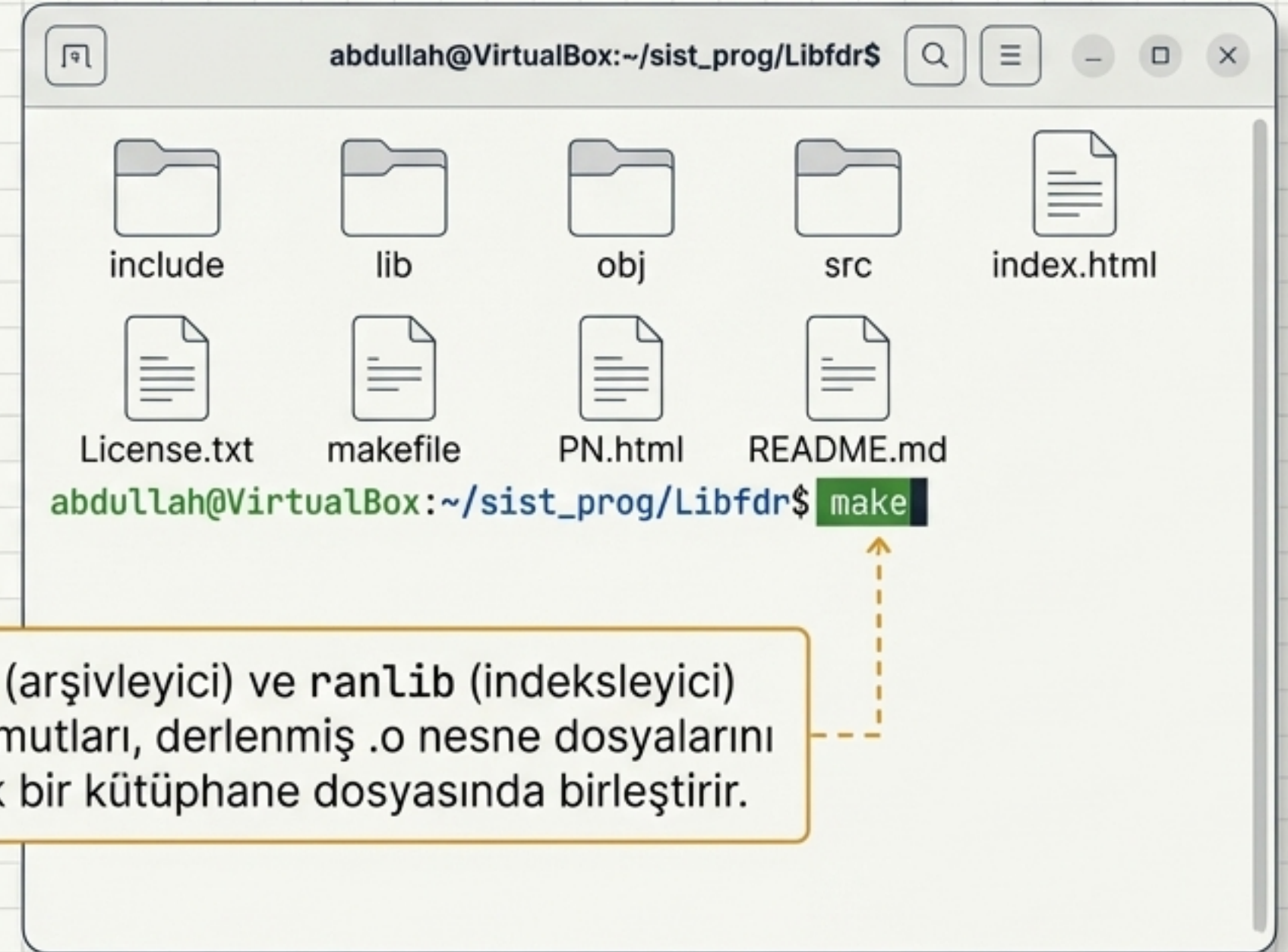
Kütüphaneyi Derlemek: Statik Bağlama

Editorial Technical Blueprint

Libfdr kaynak kodunu kullanıma hazır bir statik kütüphaneye (libfdr.a) dönüştürmek için terminalde make komutu çalıştırılır.

```
lib/libfdr.a: $(OBJJS)
  ar ru lib/libfdr.a $(OBJJS)
  ranlib lib/libfdr.a
```

ar (arşivleyici) ve ranlib (indeksleyici) komutları, derlenmiş .o nesne dosyalarını tek bir kütüphane dosyasında birleştirir.



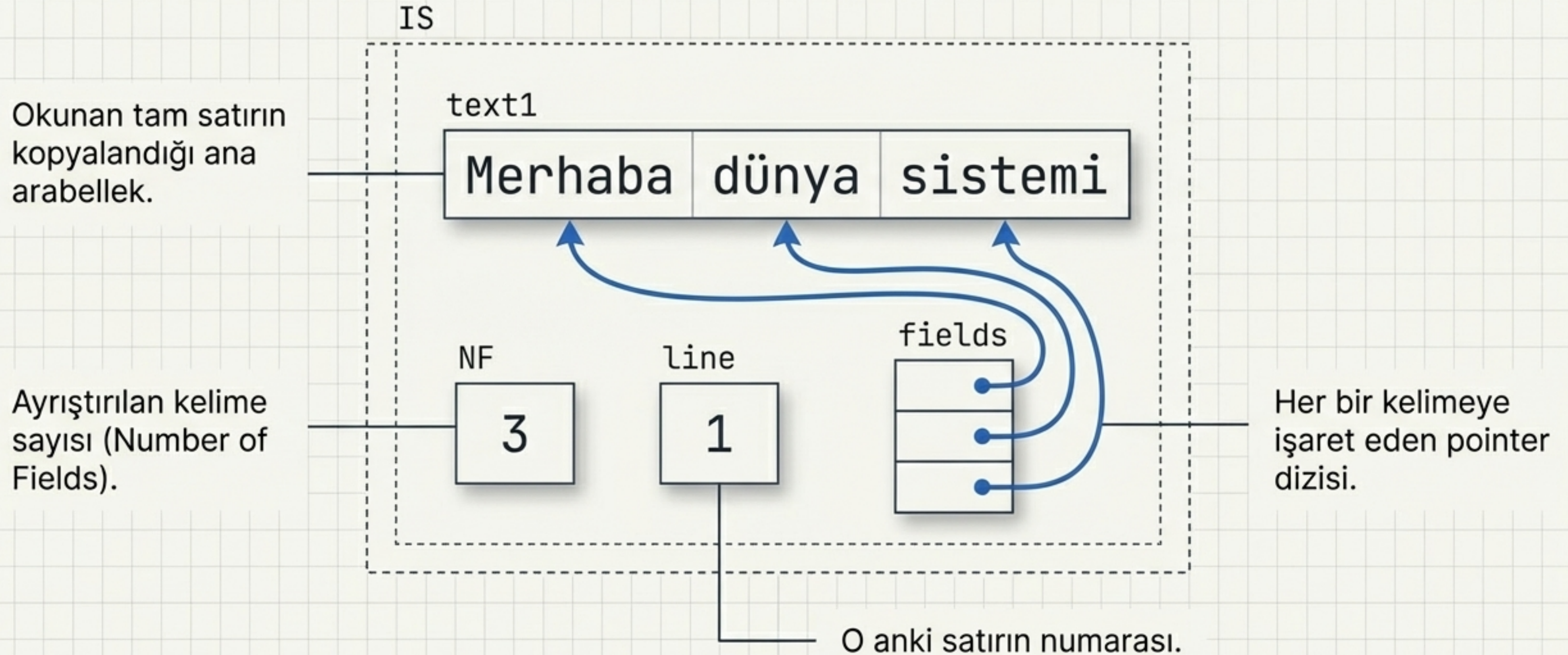
Girdi İşleme Karşılaştırması: Standart C vs. Fields

Özellik	Standart I/O (fgets / scanf)	Fields Kütüphanesi
Satır Okuma	Manuel arabellek yönetimi gerektirir.	Otomatik, güvenli okuma.
Kelime Ayırıştırma	strtok veya sscanf ile manuel ve zahmetli.	Boşluklara göre otomatik ayırıştırma.
Satır Sayacı	Geliştirici tarafından manuel takip edilir.	<code>is->line</code> ile otomatik takip.
Dosya Akışları	<code>stdin</code> , <code>stdout</code> , <code>stderr</code> kanalları.	Tek bir <code>inputstruct (IS)</code> üzerinden yönetim.

Sonuç: Fields, soyutlama sağlayarak hata payını düşürür.

inputstruct (IS) Anatomisi

```
#include <fields.h>
```



Dosya Okuma Yaşam Döngüsü



`new_inputstruct(filename)`

Dosyayı açar ve IS yapısı için malloc() ile bellek tahsis eder. (Standart girdi için NULL kullanılır).

`get_line(is)`

IS'nin durumunu değiştirir: text1'i doldurur, kelimelere ayırır, NF ve line değerlerini günceller. EOF için -1 döndürür.

`jettison_inputstruct(is)`

Dosyayı kapatır ve tahsis edilen belleği serbest bırakır.

Uygulama: Her Kelimeyi Satır Numarasıyla Yazdırmak

```
is = new_inputstruct(argv[1]);  
  
while(get_line(is) >= 0) {  
    for (i = 0; i < is->NF; i++) {  
        printf("%d: %s\n", is->line, is->fields[i]);  
    }  
}  
  
jettison_inputstruct(is);
```

Döngü, dosya sonuna kadar okumaya devam eder.

Dizi mantığı ile kelimelere doğrudan erişim.

× Terminal

```
UNIX> bin/printwords txt/rex-1.txt  
1: June:  
1: Hi  
1: ...  
1: I  
1: missed  
1: you!  
2: Rex:
```


Kritik Uyarı: `get_line()` Bellek Tahsis Etmez!

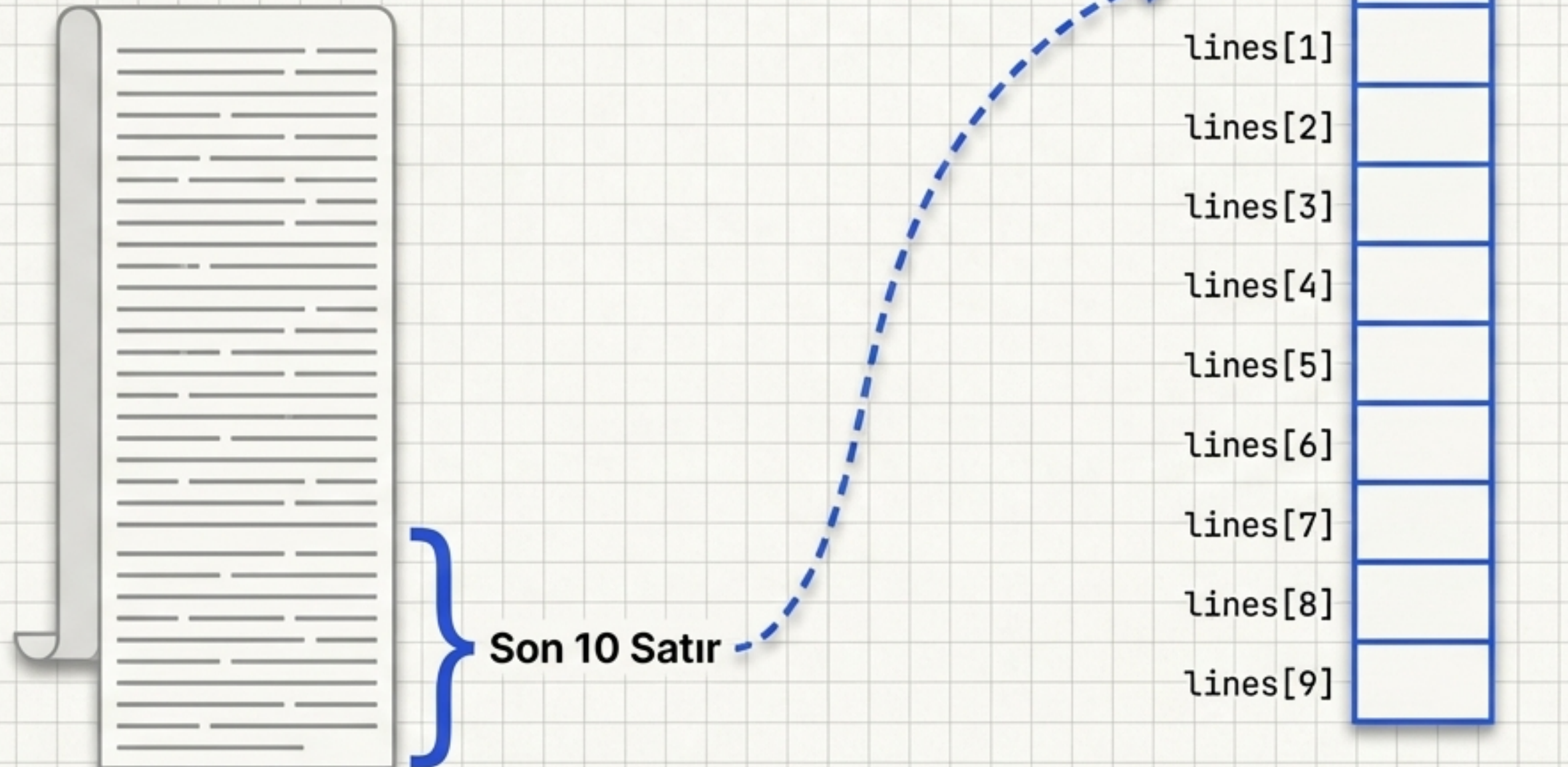


Fields kütüphanesi ile ilgili en kritik kural şudur: `malloc()` yalnızca başlangıçta `new_inputstruct()` sırasında çağrılır.

`get_line()` fonksiyonu **her çağrıldığında yeni bellek ayırmaz;** **mevcut IS yapısının alanlarının ÜZERİNE YAZAR.** Eski verilere ihtiyacınız varsa, kopyalamayı (örn. `strdup()` ile) **SİZ** yapmalısınız.

Zorluk: Son 10 Satır Yazdırmak (tail10)

Amacımız, standart girdiden okunan son 10 satır bellekte tutup program sonunda yazdırmaktır. Bunun için 10 elemanlı bir pointer dizisi (`char *lines[10]`) kullanacağız ve modüler aritmetik ($n\%10$) ile diziyi bir döngüsel arabellek (ring buffer) olarak yöneteceğiz.



Deneme 1: Saf Yaklaşım (tail10-bad.c)

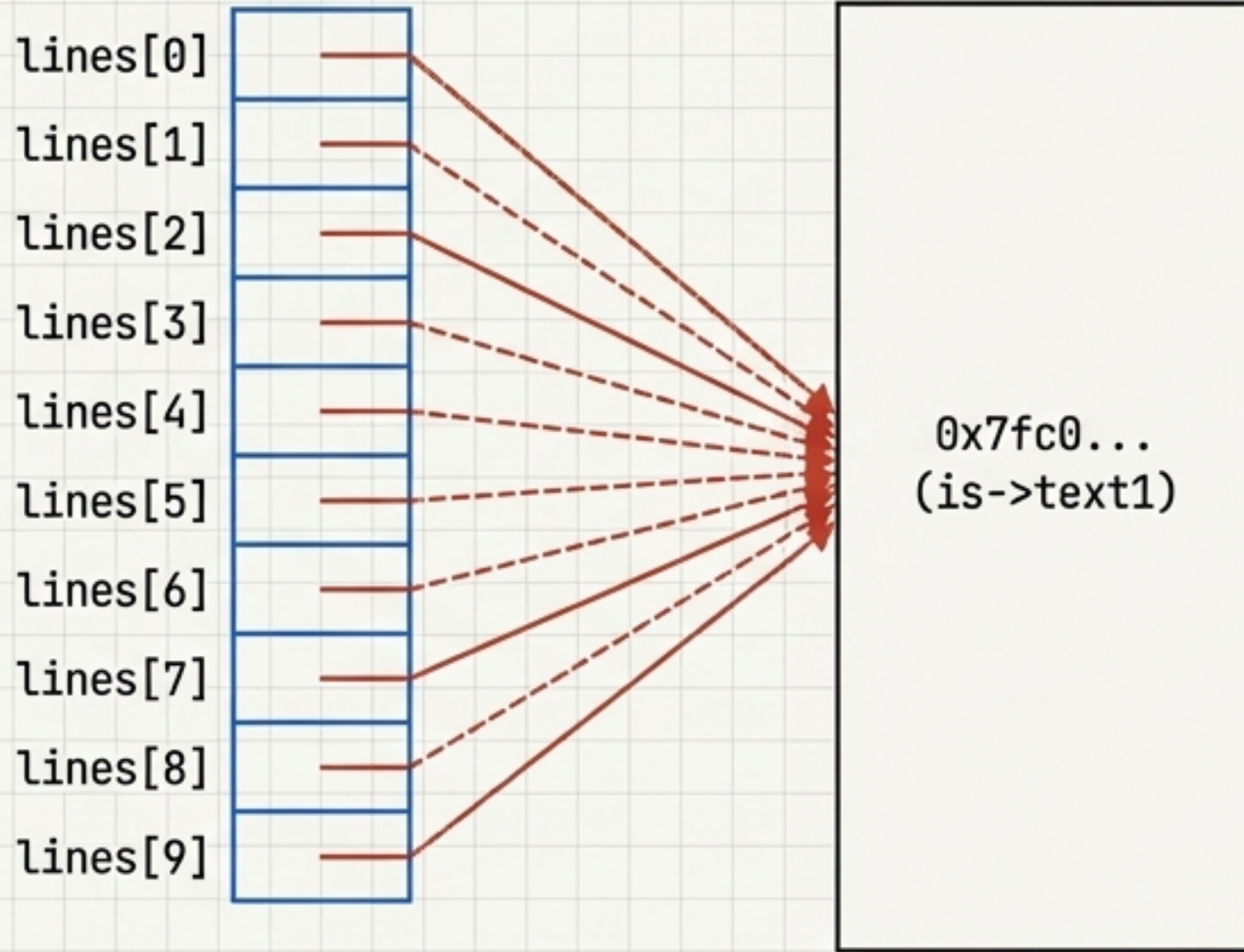
Her satırı okuduğumuzda, dizi işaretçimizi doğrudan `is->text1` arabelleğine yönlendiriyoruz. Dizeyi kopyalamıyoruz, sadece adresini atıyoruz.

```
is = new_inputstruct(NULL);  
n = 0;  
while (get_line(is) >= 0) {  
    lines[n%10] = is->text1;  
    n++;  
}
```

FATAL ERROR: Sadece adres kopyalanıyor, verinin kendisi değil!

Pointer Aliasing (İşaretçi Örtüşmesi) Tuzağı

The Cause



Tüm dizi elemanları aynı bellek adresini gösterir.
`get_line()` her çalıştığında bu tek arabelleğin üzerine yazar.

The Effect

```
UNIX> bin/tail10-bad < txt/tail-input-15.txt
15  Sofia Godlike
15  Sofia Godlike
15  Sofia Godlike
15  Sofia Godlike
(... 10 kez tekrar eder)
15  Sofia Godlike
15  Sofia Godlike
15  Sofia Godlike
15  Sofia Godlike
15  Sofia Godlike
15  Sofia Godlike
15  Sofia Godlike
15  Sofia Godlike
15  Sofia Godlike
```

Sonuç: Dizideki tüm işaretçiler en son okunan (15.)
satırı gösterir.

Deneme 2: Kopyalama ve Bellek Sızıntısı (tail10-memory-leak.c)

Çözüm olarak strdup() kullanıyoruz. Bu fonksiyon dahili olarak malloc() çağırır, satırın tam bir kopyası için yeni bellek ayırır. Pointer örtüşmesi çözüldü, ancak yeni bir gizli tehlike doğdu.

```
while (get_line(is) >= 0) {  
    lines[n%10] = strdup(is->text1);  
    n++;  
}
```

lines[0]



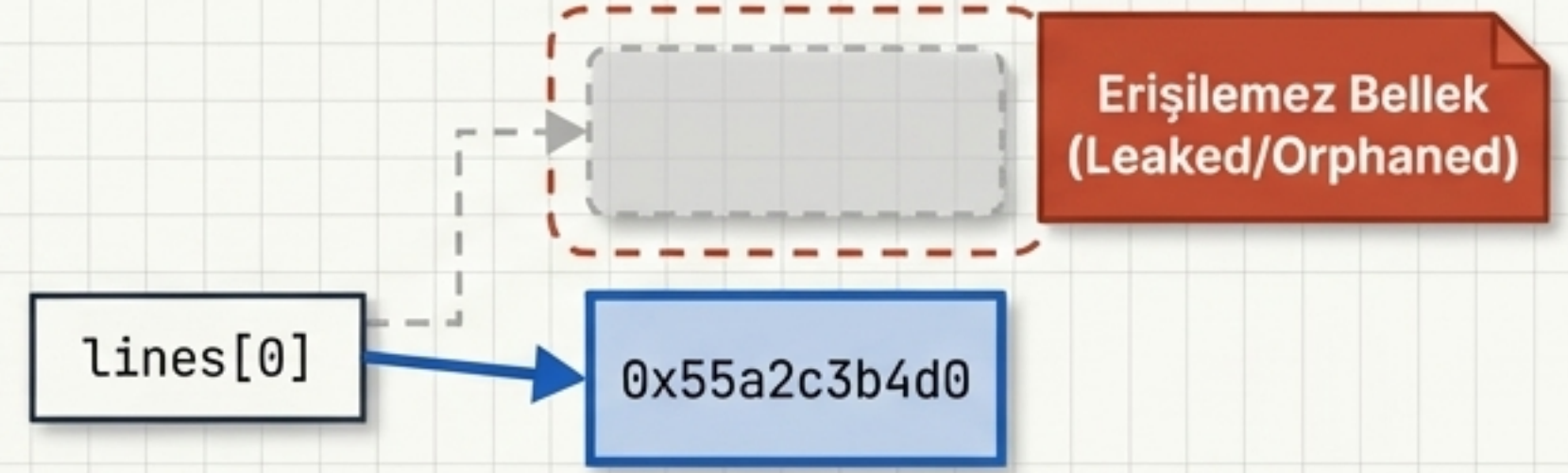
Yeni Tahsis Edilen
Bellek Bloğu



strdup() her satır için izole bir kutu (clone) oluşturur.

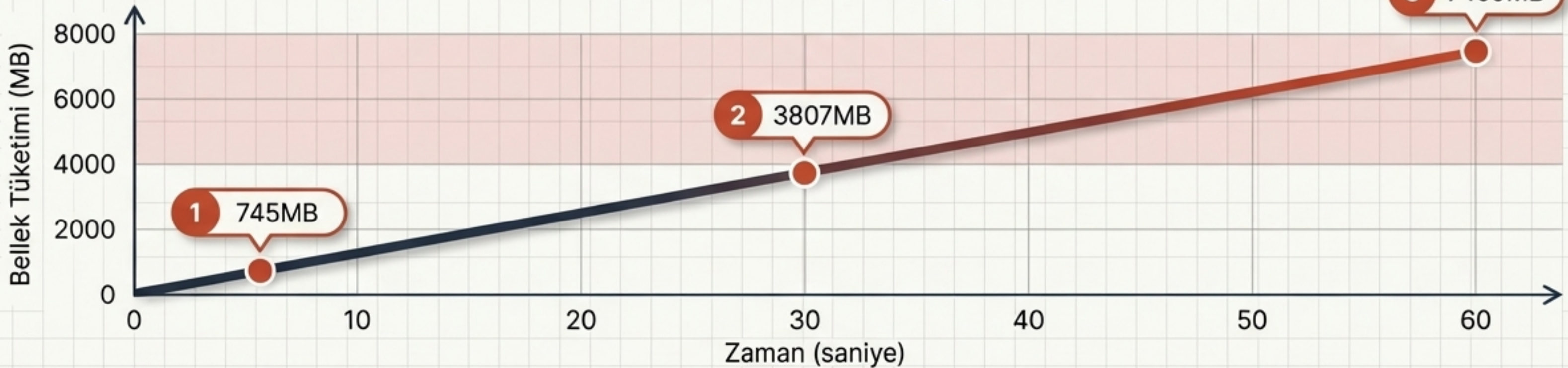
Görünmez Tehlike: Bellek Sızıntısının Anatomisi

Döngü Başına Bellek Sızıntısı



n >= 10 olduğunda, strdup() eski işaretçinin üzerine yazar.
Eski bellek serbest bırakılmadığı için yetim kalır.

Sistem Etkisi (Sonsuz Döngü Testi)

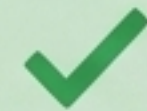


Sonsuz girdi ile test edildiğinde, program tüm sistem belleğini tüketene kadar şişer.

Deneme 3: Kusursuz Yürütme (tail10-good.c)

Denge sağlandı. strdup() ile yeni bellek ayırmadan hemen önce, dizinin o konumunda tutulan ve artık ihtiyaç duyulmayan eski belleği **free()** ile sisteme iade ediyoruz.

```
while (get_line(is) >= 0) {  
    if (n >= 10) free(lines[n%10]);  
    lines[n%10] = strdup(is->text1);  
    n++;  
}
```



Bu satır bellek sızıntısını önler.

Sistem Belleği Tüketimi (60 saniye sonra): **Sabit 492K**

Üç Yaklaşımın Analizi

Yaklaşım	Bellek Durumu	Sistem Sonucu
Saf Atama (is->text1)	 Üzerine Yazılan (Aliased)	Çöp Çıktı (Aynı satırın 10 kez tekrarı)
Yalnızca strdup()	 Yetim / Sızıntı (Leaked)	Sistemin Kilitlenmesi / RAM Tükenmesi
free() + strdup()	 Temiz (Yönetilen)	Kusursuz Çalışma

Sonuç: Kütüphane pointer sağladığında, belleğin mülkiyeti ve yaşam döngüsü sorumluluğu tamamen programcıya aittir.

Aracı Genelleştirmek: Dinamik Boyutlandırma (tailanyf.c)

Artık n değeri sabit 10 değil; komut satırından alınarak istenilen sayıda son satırı tutacak esnek bir yapı kurulmuştur.

```
if (sscanf(argv[1], "%d", &n) == 0 || n <= 0) {  
    fprintf(stderr, "usage: tailany1 n\n");  
    exit(1);  
}
```

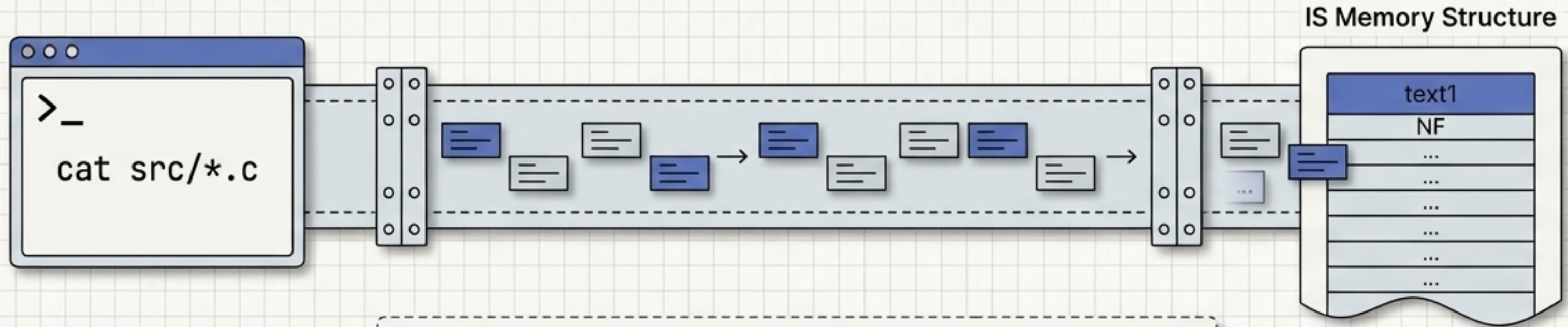
Annotation 1 (sscanf): Kullanıcı argümanlarını güvenle tamsayıya dönüştürmek ve test etmek.

Annotation 2 (malloc): Çalışma zamanına kadar boyutu bilinmeyen dinamik pointer dizileri oluşturmak.

```
lastn = (char **) malloc(sizeof(char *) * n);  
if (lastn == NULL) { perror("malloc"); exit(1); }
```


Ufku Genişletmek: Shell Komutlarını Okumak

`pipe_inputstruct()` fonksiyonu, `popen()` kullanarak bir sistem komutunun standart çıktısını doğrudan bir IS yapısına bağlamanıza olanak tanır.

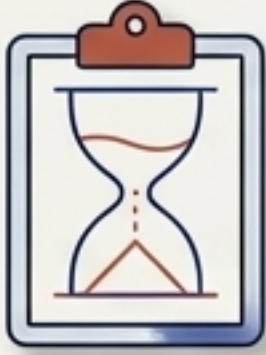


```
is = pipe_inputstruct("cat src/*.c");  
while (get_line(is) >= 0) nlines++;  
printf("# lines in src/*.c: %d\n", nlines);
```

```
# lines in src/*.c: 226
```

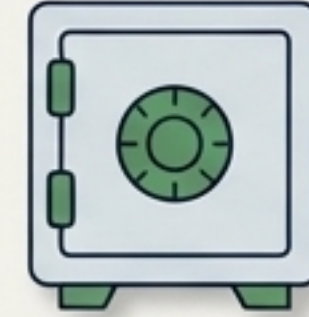

Sentez: Kalıcılık Kuralı (The Rule of Persistence)

Kütüphanenin Sorumluluğu (Şimdi / The Present)



Fields kütüphanesi sadece '**Şu An**' ile ilgilenir. Mevcut satırı okur, böler ve **geçici bir tamponda** sunar. Gelecek döngülere karşı hafızası yoktur.

Programcının Sorumluluğu (Gelecek / The Future)



Verinin geleceğe taşınması gerekiyorsa, **Kalıcılık Kuralı** devreye devreye girer. Geliştirici, veriyi **kopyalamak** (strdup/malloc) ve yaşam döngüsü bittiğinde **yok etmekten** (free) sorumludur.



Nihai Görev

- > # Öğrenilen kavramları birleştirin:
- > # İstenilen dosyadan okunan son 'n' satırdaki
- > # 'm' adet kelimeyi yazdıran C kodunu geliştirin.
- > █

İpucu: Pointer dizileri, `get_line()` kelime ayrıştırması (NF özelliği) ve kusursuz bir `free()` / `strdup()` döngüsünü birlikte kullanmanız gerekecek.